NetraAI

An Intelligent Image and Video Processing Agent for Assistive Vision



using free and open-source tools on Ubuntu (WSL2)

Local LLM tool calling · OpenCV · YOLO · OCR · speech · edge deployment

Tulja

Technical tutorial · November 2026

Companion code: `https://github.com/<your-user>/netra-ai`

Abstract

This tutorial shows how to design, install, program, test and publish a locally running AI agent for image and video processing in an *assistive-vision* setting: a camera worn by, or carried with, a person with visual impairment, and software that turns frames into short spoken information. A local large language model (LLM) served by Ollama interprets natural-language instructions and selects tools from a fixed allow-list. Established libraries perform the numerical work: OpenCV and scikit-image for enhancement, filtering and metrics; a small YOLO detector for objects; Tesseract for printed text; espeak-ng for speech. The tutorial covers WSL2/Ubuntu setup, the tool library, a bounded agent loop, a left/center/right and proximity guidance layer, controlled experiments, evaluation metrics, a Streamlit interface, edge-deployment considerations for a Raspberry Pi-class device, safety and privacy practice, and step-by-step publication to GitHub. All code is in the companion repository and is included in this document by reference, so the text and the code cannot drift apart.

Contents

1 Introduction **4**

1.1 What is an assistive-vision agent? . . . 4
1.2 Why this domain . . . 4
1.3 Example requests . . . 4
1.4 Architecture . . . 5
1.5 Why a local, open-source stack? . . . 5
1.6 What you will build . . . 5

2 Software Stack and Responsibilities **7**

3 Ubuntu Development Environment **8**

3.1 Windows users: install WSL2 with Ubuntu . . . 8
3.2 Update package information . . . 8
3.3 Install system dependencies . . . 9
3.4 Verify the installations . . . 9
3.5 Get the project . . . 9
3.6 Create and activate a virtual environment . . . 10
3.7 Install Python libraries . . . 10
3.8 Run the test suite . . . 10

4 Installing and Configuring Ollama **11**

4.1 Install Ollama . . . 11
4.2 Check the service . . . 11
4.3 Download a language model . . . 11
4.4 Test the model and the API . . . 11

5 Image-Processing Tools **13**

5.1 Shared helpers: safe paths and limits . . . 13
5.2 Image quality assessment . . . 14
5.3 Enhancement: CLAHE on lightness, then gamma . . . 15
5.4 Filtering, sharpening, edges and segmentation . . . 15
5.5 Further OpenCV operations . . . 19

6 Video-Processing Tools **20**

6.1 Properties, sampling and motion . . . 20
6.2 FFmpeg from the terminal . . . 22

7 The Assistive Layer: Detection, Guidance, Text and Speech **23**

7.1 From detections to guidance . . . 23
7.1.1 The rules . . . 23
7.2 Object detection with a small YOLO model . . . 25
7.3 Reading printed text . . . 28
7.4 Speech output . . . 29
7.5 Scene description with a vision-language model . . . 30
7.6 A deterministic pipeline . . . 30

8 Connecting the Tools to the AI Agent 32
8.1 Function calling . . . 32
8.2 The agent controller . . . 32
8.3 Run the agent . . . 34
8.4 Hardening checklist . . . 35

9 Practical Experiments 36
9.1 Controlled low-light experiment (measured) . . . 36

10 Evaluation 38
10.1 Image-quality metrics . . . 38
10.2 Detection metrics . . . 38
10.3 Task-level and system-level measures . . . 40
10.4 Testing and reproducibility . . . 40

11 Interfaces: Streamlit and the Live Loop 42
11.1 Streamlit . . . 42
11.2 Live guidance from a camera or a video file . . . 43

12 Edge Deployment 45
12.1 Design levers . . . 45
12.2 Exporting a model . . . 45
12.3 On the Raspberry Pi . . . 45

13 Reliability, Safety, Privacy and Ethics 46
13.1 Failure modes and safeguards . . . 46
13.2 Operational rules . . . 46
13.3 Privacy . . . 46
13.4 Working with users . . . 46

14 Publishing the Project on GitHub 47
14.1 Before the first commit . . . 47
14.2 Create the local repository . . . 47
14.3 Create the GitHub repository . . . 47
14.4 Authenticate . . . 47

14.5 Push 48
14.6 Good practice afterwards 48
14.7 Building this document 48

15 Roadmap and Hardware **49**

16 Official Documentation and Further Reading **50**

Conclusion **51**

1 Introduction

1.1 What is an assistive-vision agent?

An *image and video processing (IVP) agent* accepts a request in natural language, decides which computer-vision operation fits, runs it, checks the result and reports what it did. For an assistive-vision project the camera is the sensor and the user is a person who cannot rely on sight alone. Typical needs are: knowing what is in front of them and roughly where, reading printed labels, and getting information in a form they can use immediately, which usually means speech rather than a picture.

The language model is the *coordinator*, not the vision engine. It never touches pixels. It reads the request, picks a function such as `enhance_low_light` or `detect_objects`, and explains the result. OpenCV, scikit-image, YOLO, Tesseract and espeak-ng do the actual computation, and their outputs are deterministic and measurable.

1.2 Why this domain

Assistive vision is a good teaching domain because it forces engineering trade-offs that generic image-processing exercises hide.

- **Latency matters.** A description that arrives three seconds late may describe a scene the user has already left.
- **Bad input is normal.** A camera on a moving head sees motion blur, low light, glare and awkward angles. Image-quality assessment and enhancement are part of the pipeline, not an optional extra.
- **Errors have asymmetric cost.** Missing a hazard is worse than a false alarm, but too many false alarms make a user stop listening. This is why evaluation here goes beyond accuracy.
- **Edge constraints are real.** The final device is typically a small single-board computer with limited memory, so model size, input resolution and frame rate become design variables.
- **Privacy is not optional.** A wearable camera records bystanders. Local processing keeps frames on the device.

Safety statement

NetraAI is a research and teaching prototype. It is **not** a certified mobility or safety device. It must not replace a white cane, a guide dog or orientation-and-mobility training. Detections are estimates; the system never announces that a path is "clear", only which known objects it detected. Any evaluation with visually impaired participants needs ethical approval, informed consent and involvement of qualified mobility instructors.

1.3 Example requests

The finished agent should handle requests such as:

- "Is `hallway.png` too dark? If so, enhance it and tell me what you changed."
- "Detect objects in `street.png` and tell me where the closest one is."

- "Read the text on `medicine_box.png` in English and Telugu."
- "Extract one frame every two seconds from `walk.mp4`."
- "Darken `astronaut.png`, enhance it, and compare PSNR and SSIM against the original."
- "Analyse `crossing.mp4` every half second and save a guidance timeline."
- "Run the full navigation pipeline on `corridor.png` and give me the audio file."

1.4 Architecture

Figure 1.1 separates five responsibilities: interaction, reasoning, controlled tool execution, deterministic vision processing, and presentation. The controller is the trust boundary. The model proposes; the controller validates and executes.

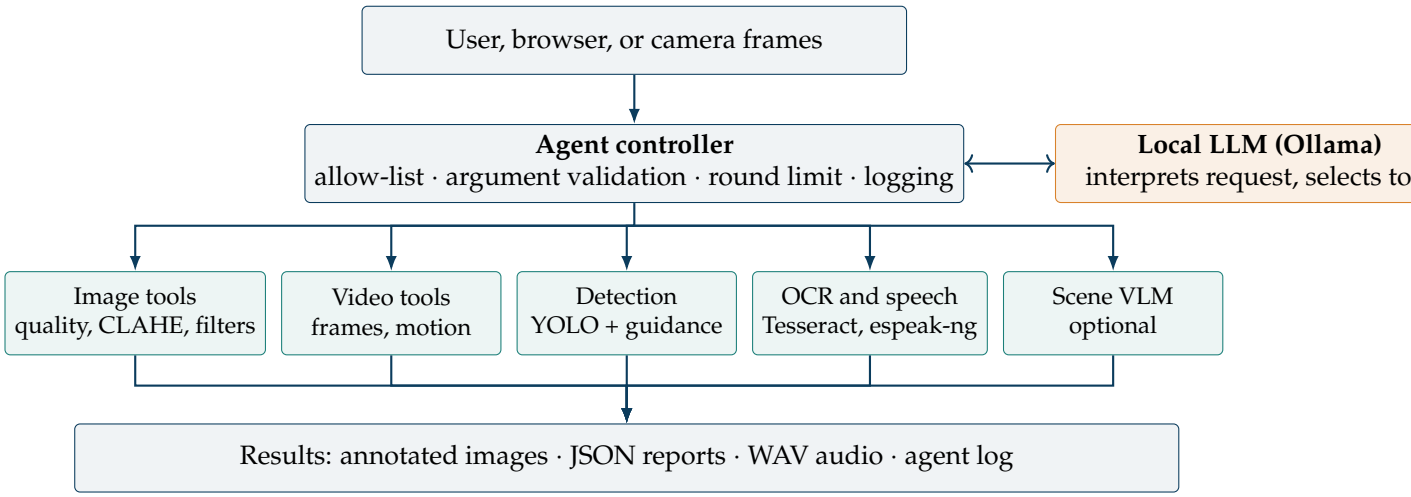


Figure 1.1: Logical architecture of NetraAI. Only functions in the allow-list can be executed.

1.5 Why a local, open-source stack?

A local system keeps camera frames on the user's computer, avoids recurring service cost, works offline after the initial downloads, and is reproducible because algorithms, parameters and versions can be recorded. "Free and open source" needs a qualification: a library may be open source while a particular *model weight*, dataset, or commercial deployment has additional conditions. This matters here because the detector used in this tutorial is distributed under a licence with obligations for distribution and network use (Chapter 7). Always read the licence of the exact version you ship.

1.6 What you will build

The companion repository has the following layout. Every listing in this document is included directly from these files.

```
netra-ai/
|-- netra/
|   |-- common.py         safe paths, limits, image I/O
|   |-- image_tools.py    quality, enhancement, filters, edges, segmentation
|   |-- video_tools.py    metadata, frame sampling, motion detection
```

```
|   |-- spatial.py       zones, proximity, guidance sentence, throttle
|   |-- detection.py     YOLO inference, annotated output, video timeline
|   |-- ocr_tools.py     Tesseract text reading
|   |-- speech.py        espeak-ng text-to-speech to WAV
|   |-- vision_tools.py  vision-language scene description
|   |-- metrics.py       PSNR, SSIM, IoU, precision/recall, low-light simulator
|   |-- pipeline.py      assess -> enhance -> detect -> speak
|   |-- agent.py         allow-listed tool-calling loop
|   `-- live.py          webcam / video-file guidance loop
|-- app.py               Streamlit interface
|-- tests/               pytest suite
|-- scripts/             setup_ubuntu.sh, run_lowlight_experiment.py
|-- docs/                this tutorial (LaTeX source and PDF)
`-- README.md, LICENSE, requirements*.txt, .gitignore, .github/workflows/ci.yml
```

Validation status of the companion code

The image tools, video tools, spatial guidance, metrics, tool schemas, allow-list behaviour and the detection post-processing (with a stand-in model) are covered by an automated test suite that passes (20 tests). The low-light experiment in Chapter 9 was actually run and its numbers are reported as measured. **Not exercised when this tutorial was prepared:** real YOLO inference, tool calling against a live Ollama model, Tesseract and espeak-ng output, webcam capture, and Raspberry Pi deployment. Treat those parts as a checklist to verify on your own machine and record your results (Chapter 10).

2 Software Stack and Responsibilities

Component	Software	Purpose
Operating system	Ubuntu 24.04 LTS (native or WSL2)	Development and execution platform.
Language	Python 3	Tool functions, agent controller, tests.
Model runtime	Ollama	Runs LLMs locally and exposes an HTTP API with tool calling.
Language model	Llama or Qwen family	Interprets requests and chooses tools.
Computer vision	OpenCV (headless build)	I/O, colour conversion, CLAHE, filters, edges, contours, background subtraction.
Analysis	NumPy, scikit-image	Array maths, PSNR and SSIM.
Plots	Matplotlib	Histograms and experiment figures.
Detection	Ultralytics YOLO (nano model)	Object classes and bounding boxes.
Text reading	Tesseract OCR	Printed text, including Indian-language packs.
Speech	espeak-ng	Offline text-to-speech to WAV.
Video	FFmpeg	Decoding, encoding, standard conversions.
Interface	Streamlit	Local browser interface.
Quality	pytest, GitHub Actions	Regression tests and continuous integration.
Publishing	Git, GitHub	Version control and sharing.

Table 2.1: Components and their roles.

OpenCV. The workhorse for conventional computer vision. This tutorial uses `imread`, `imwrite`, `cvtColor`, `createCLAHE`, `LUT`, `medianBlur`, `GaussianBlur`, `bilateralFilter`, `Canny`, `threshold`, `Laplacian`, `VideoCapture`, `VideoWriter` and `createBackgroundSubtractorMOG2`. The *headless* wheel (`opencv-python-headless`) is used because it has no GUI dependencies, which avoids the shared-library errors that the standard wheel can produce on servers and in WSL.

FFmpeg. Better than OpenCV for standard media transformations (resize, trim, re-encode) and for timestamp-accurate work. OpenCV is convenient when you need custom frame-by-frame algorithms.

Ollama. Provides a local model runtime and an HTTP API. The agent sends a prompt and a list of tools; the model may answer with a tool call; the Python controller validates and executes it, then returns the result to the model.

YOLO, Tesseract, espeak-ng. These are the three assistive components: *what* is in the frame, *what the text says*, and *how to say it aloud*.

3 Ubuntu Development Environment

This chapter targets Ubuntu 24.04 LTS, either installed natively or running under Windows through WSL2. Ubuntu-specific steps are the same in both.

3.1 Windows users: install WSL2 with Ubuntu

In an administrator PowerShell window:

```
wsl --install -d Ubuntu-24.04
```

- `wsl--install` enables the WSL features and installs a Linux distribution; `-d` selects the distribution.
- Restart if asked, then create a Linux username and password in the Ubuntu window that opens.

Verify and update:

```
wsl -l -v
wsl --update
```

`-l-v` lists installed distributions with their WSL version; the Ubuntu entry should show version 2.

WSL practicalities

- **Work inside the Linux filesystem** (`~/`), not under `/mnt/c/`. File access across the boundary is much slower, which hurts video processing.
- **systemd.** Ollama's installer sets up a service. On recent WSL, Ubuntu enables systemd by default; if `systemctlstatus` reports that it is not running, add `[boot]` and `systemd=true` to `/etc/wsl.conf`, then run `wsl--shutdown` from PowerShell and reopen Ubuntu.
- **GPU.** Install the NVIDIA driver on Windows (not inside Ubuntu); CUDA-enabled tools in WSL use it. Check with `nvidia-smi` inside Ubuntu. CPU is enough for this tutorial.
- **Audio.** WSLg normally provides audio. Play a generated WAV with `paplayfile.wav` after installing `pulseaudio-utils`.
- **Webcam.** A USB camera can be forwarded with `usbipd-win`, but the default WSL kernel may lack camera (UVC) drivers. The simplest route is to develop with recorded video files in WSL and run the live camera loop on native Ubuntu or the Raspberry Pi. Check Microsoft's current WSL documentation before investing time here.

3.2 Update package information

```
sudo apt update
sudo apt upgrade -y
```

`sudo` runs a command as administrator; `apt` is Ubuntu's package manager; `update` refreshes the package index; `upgrade` installs newer versions of installed packages; `-y` answers yes to prompts (review changes first on a production machine).

3.3 Install system dependencies

```
sudo apt install -y python3 python3-pip python3-venv python3-dev \
    build-essential git curl wget ffmpeg \
    tesseract-ocr tesseract-ocr-tel tesseract-ocr-hin espeak-ng
```

The trailing backslash continues the command on the next line.

`python3, -pip, -venv, -dev`
: Interpreter, package installer, virtual environments, and headers for compiled extensions.

`build-essential`
: Compiler and build tools used by some Python packages.

`git, curl, wget`
: Version control and download tools (curl is also used by the Ollama installer).

`ffmpeg`
: Multimedia decoding and encoding.

`tesseract-ocr`
: The OCR engine; `-tel` and `-hin` add Telugu and Hindi language data, which are useful for signs and labels in India.

`espeak-ng`
: Offline speech synthesis with many languages and voices.

The same list is automated in `scripts/setup_ubuntu.sh`.

3.4 Verify the installations

```
python3 --version
ffmpeg -version
git --version
tesseract --list-langs
espeak-ng --version
```

Each prints a version or list. Recording versions is part of reproducible research.

3.5 Get the project

Either clone the finished repository, or create the structure yourself.

```
git clone https://github.com/<your-user>/netra-ai.git
cd netra-ai
mkdir -p input/images input/videos output models
```

`mkdir -p` creates directories and any missing parents without error if they exist. `input/` holds source media, `output/` holds everything generated; keeping them separate preserves originals and makes the workflow auditable.

3.6 Create and activate a virtual environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
```

-mvenv runs Python's environment module and creates the isolated .venv directory. source runs the activation script in the current shell so that python and pip refer to the environment; the prompt gains a (.venv) prefix. python-mpip guarantees that pip belongs to the active interpreter. Leave the environment with deactivate.

3.7 Install Python libraries

```
numpy
opencv-python-headless
scikit-image
matplotlib
pillow
pandas
pytesseract
ollama
streamlit
pytest
```

Listing 3.1: requirements.txt

```
pip install -r requirements.txt
```

-r reads package names from a file. After the first successful install, record exact versions with pipfreeze>requirements.lock.txt for reproducibility.

Caution

Install *only one* OpenCV wheel variant (opencv-python, opencv-python-headless or opencv-contrib-python) in an environment. Mixing them causes confusing import errors.

3.8 Run the test suite

```
python -m pytest -q
```

-q gives compact output. All tests should pass before you continue. They use a temporary project directory (set through the NETRA_HOME environment variable in tests/conftest.py), so they never touch your real images.

4 Installing and Configuring Ollama

4.1 Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

- `-f` fail on HTTP errors; `-s` silence the progress meter; `-S` still show errors; `-L` follow redirects.
- The pipe | sends the downloaded script to `sh`, which executes it.

This runs a remote script. On a managed or security-sensitive computer, download it first, read it, and follow the official installation documentation. Verify with `ollama--version`.

4.2 Check the service

```
systemctl status ollama
sudo systemctl start ollama
sudo systemctl enable ollama
```

`status` shows whether the service is active; `start` starts it now; `enable` starts it at boot. The API listens on port 11434. Do not expose this port to an untrusted network.

4.3 Download a language model

```
ollama pull llama3.1:8b
```

`pull` downloads the named model; `8b` is the approximate parameter count. Memory needs depend on quantisation and context length. Pick one model first. An alternative is `qwen3:8b`. Check the model's licence and tool-calling capability before institutional or commercial use. To use another model without editing code, set an environment variable:

```
export NETRA_LLM="qwen3:8b"
```

4.4 Test the model and the API

```
ollama run llama3.1:8b
curl http://localhost:11434/api/tags
```

`run` opens an interactive session (leave with `/bye`); the `/api/tags` endpoint returns the locally available models as JSON. A minimal Python check:

```
from ollama import chat
reply = chat(model="llama3.1:8b",
             messages=[{"role": "user", "content": "Explain CLAHE in two sentences."
    }])
print(reply.message.content)
```

The model name must match one that is installed locally.

5 Image-Processing Tools

Every tool is a plain Python function that validates its input, performs one operation, saves a *new* file and returns a small dictionary. Small, single-purpose functions are easy for a language model to choose between and easy for you to test.

5.1 Shared helpers: safe paths and limits

```python
"""Shared paths, limits and safe file helpers for NetraAI.

Every tool reads from ``input/`` and writes to ``output/``. Tools accept
plain filenames only, never paths, so a model-generated argument cannot
escape the project directories.
"""
import os
from pathlib import Path

import cv2

BASE = Path(os.getenv("NETRA_HOME", Path(__file__).resolve().parents[1])).resolve()

IMAGE_IN = BASE / "input" / "images"
VIDEO_IN = BASE / "input" / "videos"
IMAGE_OUT = BASE / "output" / "images"
VIDEO_OUT = BASE / "output" / "videos"
FRAME_OUT = BASE / "output" / "frames"
REPORT_OUT = BASE / "output" / "reports"
AUDIO_OUT = BASE / "output" / "audio"

for _d in (IMAGE_IN, VIDEO_IN, IMAGE_OUT, VIDEO_OUT, FRAME_OUT, REPORT_OUT,
    AUDIO_OUT):
    _d.mkdir(parents=True, exist_ok=True)

IMAGE_EXT = {".png", ".jpg", ".jpeg", ".bmp"}
VIDEO_EXT = {".mp4", ".avi", ".mov", ".mkv"}

MAX_PIXELS = 25_000_000      # reject very large images
MAX_VIDEO_SECONDS = 600      # reject very long videos

LLM_MODEL = os.getenv("NETRA_LLM", "llama3.1:8b")
VISION_MODEL = os.getenv("NETRA_VISION", "llama3.2-vision")

FOLDERS = {"input": IMAGE_IN, "output": IMAGE_OUT}


def safe_path(directory: Path, filename: str, allowed: set) -> Path:
    """Return directory/filename after validating name, extension and existence."""
    if not isinstance(filename, str) or Path(filename).name != filename:
        raise ValueError("Only plain filenames are accepted.")
    if Path(filename).suffix.lower() not in allowed:
        raise ValueError(f"Unsupported file type. Allowed: {sorted(allowed)}")
    path = directory / filename
    if not path.is_file():
        raise FileNotFoundError(filename)
    return path


def load_image(filename: str, folder: str = "input", grayscale: bool = False):
```

```
    """Decode an image from the input or output folder (BGR or grayscale)."""
    if folder not in FOLDERS:
        raise ValueError("folder must be 'input' or 'output'.")
    path = safe_path(FOLDERS[folder], filename, IMAGE_EXT)
    flag = cv2.IMREAD_GRAYSCALE if grayscale else cv2.IMREAD_COLOR
    img = cv2.imread(str(path), flag)
    if img is None:
        raise ValueError("Unable to decode image.")
    h, w = img.shape[:2]
    if h * w > MAX_PIXELS:
        raise ValueError(f"Image too large ({w}x{h}); limit is {MAX_PIXELS} pixels."
    )
    return img


def save_image(img, name: str) -> Path:
    """Write an image into output/images and verify that the write succeeded."""
    out = IMAGE_OUT / name
    if not cv2.imwrite(str(out), img):
        raise IOError(f"Could not save {name}.")
    return out


def rel(path) -> str:
    """Path relative to the project root, for readable tool results."""
    return str(Path(path).resolve().relative_to(BASE))
```

Listing 5.1: `netra/common.py`

- `BASE` is the project root, overridable with `NETRA_HOME`. Tests use that override.
- `safe_path` rejects anything that is not a plain filename. A model could emit `../../etc/passwd`; `Path(filename).name!=filename` stops path traversal. It also restricts extensions and checks existence.
- `load_image` decodes with OpenCV (colour images arrive in **BGR** order, not RGB), refuses oversized images, and fails loudly on undecodable files rather than returning `None`.
- `save_image` checks the Boolean returned by `imwrite`; an agent must not report success for a failed save.
- `rel` returns project-relative paths so results stay readable and do not leak absolute locations.

5.2 Image quality assessment

Before enhancing anything, measure. For an 8-bit grayscale image I of N pixels, the tool computes the mean μ, standard deviation σ, the entropy of the normalised histogram $p(k)$, and a sharpness score:

$$H = -\sum_{k=0}^{255} p(k)\log_2 p(k), \qquad S = \mathrm{Var}(\nabla^2 I),$$

where ∇^2 is the Laplacian. A sharp image has strong second derivatives at edges, so S is large; a blurred image has small S. The brightness rule flags `low_light` when $\mu < 80$ and `overexposed` when $\mu > 180$; `low_contrast` when $\sigma < 35$; `possibly_blurry` when $S < 100$.

Heuristics, not truths

These thresholds are starting points. S depends on image resolution and scene content (a plain wall is "blurry" by this measure). Tune all four on frames from your own camera, and keep the `note` field in the tool result so the model does not overstate certainty.

5.3 Enhancement: CLAHE on lightness, then gamma

Global histogram equalization maps intensities through the cumulative distribution:

$$s_k = (L-1)\sum_{j=0}^{k} p_r(r_j), \qquad L = 256.$$

It is simple but can over-amplify noise and wash out scenes with both dark and bright regions. *Contrast Limited Adaptive Histogram Equalization* (CLAHE) works on small tiles and clips each tile's histogram at a limit, which restrains noise amplification.

Applying CLAHE to B, G and R separately shifts colours. The tool instead converts to the CIE **LAB** space, applies CLAHE to the lightness channel L only, and leaves the colour channels a, b untouched. A *gamma* correction then brightens or darkens through a lookup table:

$$L_{\text{out}} = 255\left(\frac{L_{\text{in}}}{255}\right)^{1/\gamma}.$$

With $\gamma > 1$ the image brightens; with $\gamma < 1$ it darkens. The agent should report `clip_limit`, `tile_grid` and `gamma` so an experiment can be reproduced.

5.4 Filtering, sharpening, edges and segmentation

- **Median** filter replaces each pixel by the median of its neighbourhood, $g(x,y) = \text{median}\{f(s,t) : (s,t) \in S_{xy}\}$. It suppresses isolated extreme values (salt-and-pepper noise) and preserves edges better than a linear blur.
- **Gaussian** filter convolves with $G(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$. It suits roughly Gaussian sensor noise but blurs edges. Kernel sizes must be odd so that the kernel has a centre pixel.
- **Bilateral** filter weights neighbours by spatial *and* intensity distance, smoothing flat regions while keeping edges.
- **Unsharp mask** sharpens with $I_{\text{out}} = (1+a)I - a\,G_\sigma * I$, where a is the amount.
- **Canny** edges: smoothing, gradient estimation, non-maximum suppression, double thresholding and hysteresis.
- **Otsu** thresholding chooses the global threshold that maximises between-class variance, so no manual value is needed; uneven illumination still defeats global thresholds.

```python
"""Image-processing tools exposed to the agent.

Each function validates its arguments, performs one operation, saves a new
file under output/images and returns a small JSON-serialisable dictionary.
"""
import cv2
import matplotlib
import numpy as np
```

```python

matplotlib.use("Agg")  # no display needed (WSL, servers, CI)
import matplotlib.pyplot as plt  # noqa: E402

from .common import (IMAGE_EXT, IMAGE_IN, IMAGE_OUT, load_image, rel,  # noqa: E402
                     safe_path, save_image)


def _odd_kernel(k: int) -> int:
    if not 3 <= k <= 31 or k % 2 == 0:
        raise ValueError("kernel_size must be an odd integer from 3 to 31.")
    return k


def image_info(filename: str) -> dict:
    """Return width, height, channels, data type and file size of an input image.

    Args:
        filename: Image filename inside input/images.
    """
    path = safe_path(IMAGE_IN, filename, IMAGE_EXT)
    img = cv2.imread(str(path), cv2.IMREAD_UNCHANGED)
    if img is None:
        raise ValueError("Unable to decode image.")
    h, w = img.shape[:2]
    return {
        "filename": filename,
        "width": w,
        "height": h,
        "channels": 1 if img.ndim == 2 else img.shape[2],
        "data_type": str(img.dtype),
        "size_bytes": path.stat().st_size,
    }


def assess_image_quality(filename: str, folder: str = "input") -> dict:
    """Measure brightness, contrast and sharpness and list likely problems.

    Args:
        filename: Image filename.
        folder: Either 'input' or 'output'.
    """
    gray = load_image(filename, folder, grayscale=True)
    mean, std = float(gray.mean()), float(gray.std())
    p = np.bincount(gray.ravel(), minlength=256) / gray.size
    p = p[p > 0]
    entropy = float(-(p * np.log2(p)).sum())
    sharpness = float(cv2.Laplacian(gray, cv2.CV_64F).var())

    issues, advice = [], []
    if mean < 80:
        issues.append("low_light")
        advice.append("enhance_low_light with gamma between 1.5 and 2.2")
    elif mean > 180:
        issues.append("overexposed")
        advice.append("enhance_low_light with gamma between 0.5 and 0.8")
    if std < 35:
        issues.append("low_contrast")
        advice.append("enhance_low_light (CLAHE)")
    if sharpness < 100:
        issues.append("possibly_blurry")
        advice.append("sharpen_image, or recapture the frame")
    return {
        "filename": filename,
```

```
        "mean_intensity": round(mean, 2),
        "std_intensity": round(std, 2),
        "entropy_bits": round(entropy, 3),
        "dark_pixel_fraction": round(float((gray < 50).mean()), 3),
        "bright_pixel_fraction": round(float((gray > 205).mean()), 3),
        "laplacian_variance": round(sharpness, 2),
        "issues": issues,
        "suggested_tools": advice,
        "note": "Thresholds are heuristics; tune them on your own camera data.",
    }


def convert_grayscale(filename: str) -> dict:
    """Convert an input image to grayscale.

    Args:
        filename: Image filename inside input/images.
    """
    gray = cv2.cvtColor(load_image(filename), cv2.COLOR_BGR2GRAY)
    return {"output": rel(save_image(gray, f"gray_{filename}"))}


def enhance_low_light(filename: str, clip_limit: float = 2.0,
                      tile_grid: int = 8, gamma: float = 1.0) -> dict:
    """Improve contrast with CLAHE on the lightness channel, then apply gamma.

    Args:
        filename: Image filename inside input/images.
        clip_limit: CLAHE clip limit, 0.1 to 10.
        tile_grid: CLAHE tiles per side, 2 to 32.
        gamma: Values above 1 brighten, below 1 darken; range 0.3 to 3.
    """
    if not 0.1 <= clip_limit <= 10:
        raise ValueError("clip_limit must be in [0.1, 10].")
    if not 2 <= tile_grid <= 32:
        raise ValueError("tile_grid must be in [2, 32].")
    if not 0.3 <= gamma <= 3:
        raise ValueError("gamma must be in [0.3, 3].")
    img = load_image(filename)
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l_ch, a_ch, b_ch = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=(tile_grid, tile_grid
    ))
    l_ch = clahe.apply(l_ch)
    if gamma != 1.0:
        table = np.array([((i / 255.0) ** (1.0 / gamma)) * 255 for i in range(256)],
                         dtype=np.uint8)
        l_ch = cv2.LUT(l_ch, table)
    out = cv2.cvtColor(cv2.merge([l_ch, a_ch, b_ch]), cv2.COLOR_LAB2BGR)
    return {"output": rel(save_image(out, f"enhanced_{filename}")),
            "parameters": {"clip_limit": clip_limit, "tile_grid": tile_grid, "gamma":
     gamma}}


def histogram_equalization(filename: str) -> dict:
    """Apply global histogram equalization to the grayscale version of an image.

    Args:
        filename: Image filename inside input/images.
    """
    gray = load_image(filename, grayscale=True)
    return {"output": rel(save_image(cv2.equalizeHist(gray), f"equalized_{filename}"
    ))}

```

```python

def denoise_image(filename: str, method: str = "median", kernel_size: int = 5) -> dict:
    """Remove noise with a median, gaussian or bilateral filter.

    Args:
        filename: Image filename inside input/images.
        method: One of 'median', 'gaussian', 'bilateral'.
        kernel_size: Odd neighbourhood size from 3 to 31.
    """
    k = _odd_kernel(kernel_size)
    img = load_image(filename)
    if method == "median":
        out = cv2.medianBlur(img, k)
    elif method == "gaussian":
        out = cv2.GaussianBlur(img, (k, k), 0)
    elif method == "bilateral":
        out = cv2.bilateralFilter(img, k, 75, 75)
    else:
        raise ValueError("method must be 'median', 'gaussian' or 'bilateral'.")
    return {"output": rel(save_image(out, f"{method}_{filename}")),
            "parameters": {"method": method, "kernel_size": k}}


def sharpen_image(filename: str, amount: float = 1.0) -> dict:
    """Sharpen an image with an unsharp mask.

    Args:
        filename: Image filename inside input/images.
        amount: Sharpening strength from 0.1 to 3.
    """
    if not 0.1 <= amount <= 3:
        raise ValueError("amount must be in [0.1, 3].")
    img = load_image(filename)
    blur = cv2.GaussianBlur(img, (0, 0), 1.5)
    out = cv2.addWeighted(img, 1 + amount, blur, -amount, 0)
    return {"output": rel(save_image(out, f"sharp_{filename}")),
            "parameters": {"amount": amount}}


def canny_edges(filename: str, low_threshold: int = 100, high_threshold: int = 200) -> dict:
    """Detect edges with the Canny algorithm.

    Args:
        filename: Image filename inside input/images.
        low_threshold: Lower hysteresis threshold, 0 to 254.
        high_threshold: Upper hysteresis threshold, above low_threshold, up to 255.
    """
    if not 0 <= low_threshold < high_threshold <= 255:
        raise ValueError("Need 0 <= low_threshold < high_threshold <= 255.")
    gray = load_image(filename, grayscale=True)
    edges = cv2.Canny(gray, low_threshold, high_threshold)
    return {"output": rel(save_image(edges, f"edges_{filename}")),
            "parameters": {"low": low_threshold, "high": high_threshold}}


def threshold_segmentation(filename: str, use_otsu: bool = True,
                           threshold_value: int = 127) -> dict:
    """Segment foreground from background with a global threshold.

    Args:
        filename: Image filename inside input/images.
        use_otsu: If true, Otsu's method chooses the threshold automatically.
```

```
        threshold_value: Manual threshold from 0 to 255 when use_otsu is false.
    """
    if not 0 <= threshold_value <= 255:
        raise ValueError("threshold_value must be in [0, 255].")
    gray = load_image(filename, grayscale=True)
    flag = cv2.THRESH_BINARY | (cv2.THRESH_OTSU if use_otsu else 0)
    used, mask = cv2.threshold(gray, threshold_value, 255, flag)
    return {"output": rel(save_image(mask, f"segmented_{filename}")),
            "parameters": {"threshold_used": float(used), "otsu": use_otsu},
            "foreground_pixels": int(np.count_nonzero(mask))}


def generate_histogram(filename: str, folder: str = "input") -> dict:
    """Plot the grayscale intensity histogram of an image.

    Args:
        filename: Image filename.
        folder: Either 'input' or 'output'.
    """
    gray = load_image(filename, folder, grayscale=True)
    hist = cv2.calcHist([gray], [0], None, [256], [0, 256]).ravel()
    fig, ax = plt.subplots(figsize=(7, 3))
    ax.plot(hist, color="#1f4e79")
    ax.set(title=f"Histogram: {filename}", xlabel="Intensity", ylabel="Pixels", xlim
    =(0, 255))
    out = IMAGE_OUT / f"histogram_{filename}.png"
    fig.savefig(out, bbox_inches="tight", dpi=150)
    plt.close(fig)
    return {"output": rel(out)}
```

Listing 5.2: `netra/image_tools.py`

Reading the code.

- Each public function has a one-line summary and an `Args:` block. The Ollama Python client converts type annotations and these docstrings into the tool description sent to the model, so the docstring is part of the interface, not decoration.
- Argument ranges are checked *inside* the tool. Never rely on the model to respect ranges.
- `matplotlib.use("Agg")` selects a non-interactive backend so plotting works without a display (WSL, servers, CI). `plt.close(fig)` releases memory in long-running agents.
- Output names are prefixed (`enhanced_`, `median_`, ...) so originals are never overwritten.

5.5 Further OpenCV operations

Operation	Function	Use
Sobel / Laplacian	`cv2.Sobel`, `cv2.Laplacian`	Gradient and second-derivative edge maps.
Erosion / dilation	`cv2.erode`, `cv2.dilate`	Shrink or grow foreground regions.
Opening / closing	`cv2.morphologyEx`	Remove specks or fill small gaps.
Contours	`cv2.findContours`	Boundaries of binary regions.
Hough transform	`cv2.HoughLines`, `cv2.HoughCircles`	Lines (kerbs, crossings) and circles.
Watershed	`cv2.watershed`	Separate touching objects from markers.

6 Video-Processing Tools

6.1 Properties, sampling and motion

For N_f frames at f frames per second the nominal duration is $T = N_f / f$. This depends on container metadata and is unreliable for variable-frame-rate video; use FFmpeg or timestamp-aware decoding when precision matters.

To sample one frame every Δt seconds, the code keeps every $\lceil f\,\Delta t \rfloor$-th frame (`round(fps*interval)`).

Motion detection uses *MOG2* background subtraction, which models each pixel as a mixture of Gaussians and flags pixels that do not fit the learned background. Its mask labels shadows with the value 127, so the code thresholds at 200 to discard them, applies a morphological opening to remove specks, and keeps only regions larger than `min_area_fraction` of the frame.

Camera motion

Background subtraction assumes a *still* camera. On a head-mounted or walking camera the whole scene moves, so almost every pixel is flagged. Use motion detection here as a baseline for static-camera clips (for example a fixed camera at a crossing), and use the object detector for wearable footage.

```python
"""Video-processing tools: metadata, frame sampling and motion detection."""
import cv2

from .common import (FRAME_OUT, MAX_VIDEO_SECONDS, VIDEO_EXT, VIDEO_IN, VIDEO_OUT,
                     rel, safe_path)


def _open(filename: str):
    path = safe_path(VIDEO_IN, filename, VIDEO_EXT)
    cap = cv2.VideoCapture(str(path))
    if not cap.isOpened():
        raise ValueError("Cannot open video.")
    fps = cap.get(cv2.CAP_PROP_FPS)
    frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    if fps <= 0:
        cap.release()
        raise ValueError("Invalid frame rate in video metadata.")
    if frames / fps > MAX_VIDEO_SECONDS:
        cap.release()
        raise ValueError(f"Video longer than {MAX_VIDEO_SECONDS} s.")
    return path, cap, fps, frames


def video_info(filename: str) -> dict:
    """Return resolution, frame rate, frame count and duration of an input video.

    Args:
        filename: Video filename inside input/videos.
    """
    _, cap, fps, frames = _open(filename)
    info = {
        "filename": filename,
        "width": int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
        "height": int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)),
        "fps": round(fps, 3),
        "total_frames": frames,
```

```
        "duration_seconds": round(frames / fps, 2),
    }
    cap.release()
    return info


def extract_frames(filename: str, interval_seconds: float = 1.0, max_frames: int =
    200) -> dict:
    """Save one frame every interval_seconds from a video.

    Args:
        filename: Video filename inside input/videos.
        interval_seconds: Sampling interval in seconds, 0.04 to 60.
        max_frames: Maximum number of frames to save, 1 to 1000.
    """
    if not 0.04 <= interval_seconds <= 60:
        raise ValueError("interval_seconds must be in [0.04, 60].")
    if not 1 <= max_frames <= 1000:
        raise ValueError("max_frames must be in [1, 1000].")
    path, cap, fps, _ = _open(filename)
    out_dir = FRAME_OUT / path.stem
    out_dir.mkdir(parents=True, exist_ok=True)
    step = max(1, round(fps * interval_seconds))
    count = saved = 0
    while saved < max_frames:
        ok, frame = cap.read()
        if not ok:
            break
        if count % step == 0:
            target = out_dir / f"frame_{saved:05d}.jpg"
            if not cv2.imwrite(str(target), frame):
                cap.release()
                raise IOError("Could not save a frame.")
            saved += 1
        count += 1
    cap.release()
    return {"frames_read": count, "frames_saved": saved, "output_directory": rel(
    out_dir),
            "parameters": {"interval_seconds": interval_seconds, "frame_step": step}}


def detect_motion(filename: str, min_area_fraction: float = 0.005) -> dict:
    """Detect moving regions with MOG2 background subtraction and draw boxes.

    Works best with a still camera; a head-mounted or walking camera moves the
    whole scene, so use this only as a baseline for static-camera clips.

    Args:
        filename: Video filename inside input/videos.
        min_area_fraction: Smallest region, as a fraction of frame area, 0.0005 to
    0.5.
    """
    if not 0.0005 <= min_area_fraction <= 0.5:
        raise ValueError("min_area_fraction must be in [0.0005, 0.5].")
    path, cap, fps, _ = _open(filename)
    w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    out = VIDEO_OUT / f"motion_{path.stem}.mp4"
    writer = cv2.VideoWriter(str(out), cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))
    if not writer.isOpened():
        cap.release()
        raise RuntimeError("Cannot create the output video (codec unavailable).")
    subtractor = cv2.createBackgroundSubtractorMOG2(history=200, varThreshold=25,
```

```
                                                    detectShadows=True)
    kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
    min_area = min_area_fraction * w * h
    total = moving = 0
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        mask = subtractor.apply(frame)
        mask = cv2.threshold(mask, 200, 255, cv2.THRESH_BINARY)[1]   # drop shadow
    label (127)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.
    CHAIN_APPROX_SIMPLE)
        found = False
        for c in contours:
            if cv2.contourArea(c) >= min_area:
                x, y, bw, bh = cv2.boundingRect(c)
                cv2.rectangle(frame, (x, y), (x + bw, y + bh), (0, 255, 0), 2)
                found = True
        moving += int(found)
        total += 1
        writer.write(frame)
    cap.release()
    writer.release()
    return {"output": rel(out), "frames_processed": total, "frames_with_motion":
    moving,
            "parameters": {"min_area_fraction": min_area_fraction}}
```

Listing 6.1: `netra/video_tools.py`

6.2 FFmpeg from the terminal

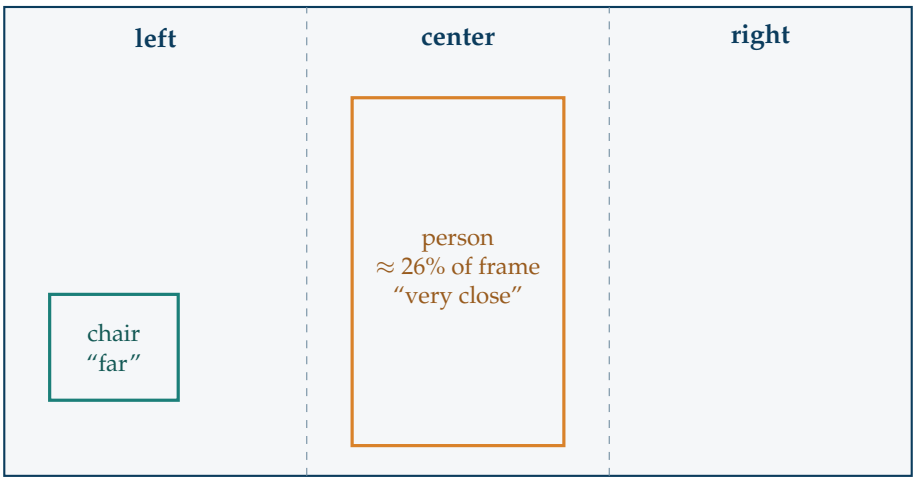
```
ffmpeg -i walk.mp4 -vf fps=1 frame_%04d.jpg          # one frame per second
ffmpeg -i walk.mp4 -vf scale=640:-2 walk_640.mp4     # resize, keep aspect ratio
ffmpeg -ss 00:00:10 -i walk.mp4 -t 10 clip.mp4       # 10 s starting at 10 s
ffmpeg -i walk.mp4 -vf format=gray walk_gray.mp4     # grayscale
```

`-i` names the input; `-vf` applies a video filter; `fps=1` samples one frame per second; `scale=640:-2` sets the width to 640 and picks an even height that preserves the aspect ratio; `-ss` seeks; `-t` limits the duration. When calling FFmpeg from Python, use `subprocess.run([...])` with a list of arguments and validated paths. Never pass model-generated text to a shell.

7 The Assistive Layer: Detection, Guidance, Text and Speech

7.1 From detections to guidance

An object detector returns class labels, confidences and pixel boxes. A blind user needs a short sentence: *what*, *which side*, and *how close*. The module `spatial.py` is deliberately free of OpenCV and model code. It is pure Python, so it is fast to test and portable to constrained devices.



Illustrative layout, not model output. Zone = box centre; proximity = box area ÷ frame area.

Figure 7.1: Guidance logic: thirds of the frame give the side; the fraction of the frame covered by the box gives a coarse proximity class.

7.1.1 The rules

- **Zone.** Let $c_x = (x_1 + x_2)/(2W)$. Left if $c_x < 1/3$, right if $c_x > 2/3$, otherwise centre.
- **Proximity.** Let $r = \dfrac{(x_2 - x_1)(y_2 - y_1)}{WH}$. "Very close" if $r \geq 0.30$, "close" if $r \geq 0.08$, otherwise "far".
- **Ranking.** Sort by proximity, then hazard classes (person, vehicles, bicycle, dog, ...), then centre before sides, then confidence. Speak at most three items.
- **No false reassurance.** With no detections the sentence is "No known objects detected. This does not mean the way is clear."

Monocular distance is crude

Box area is a poor distance estimate: a bus far away and a chair nearby can cover the same fraction of the frame. The thresholds above are placeholders for a demonstration. For real use, calibrate against tape-measured distances per class (Chapter 10), restrict claims to the classes you calibrated, or add a depth sensor (stereo camera, ToF, or ultrasonic).

```
"""Turn raw detections into short spoken guidance.
```

```
Pure Python (no OpenCV, no model) so it is fast to test and easy to port to
a microcontroller-class device later.
"""
import time

HAZARDS = {"person", "bicycle", "car", "motorcycle", "bus", "truck", "dog",
           "stop sign", "traffic light", "fire hydrant", "bench", "chair"}
PROX_RANK = {"very close": 3, "close": 2, "far": 1}
PHRASE = {"left": "on your left", "center": "ahead", "right": "on your right"}


def horizontal_zone(x1: float, x2: float, width: float) -> str:
    """Split the frame into left / center / right thirds by box centre."""
    cx = (x1 + x2) / 2 / width
    if cx < 1 / 3:
        return "left"
    if cx > 2 / 3:
        return "right"
    return "center"


def proximity(x1, y1, x2, y2, width, height) -> str:
    """Rough closeness from the fraction of the frame the box covers.

    This is a crude monocular proxy. It depends on object size and camera
    field of view; calibrate it, or add a depth sensor, before relying on it.
    """
    ratio = ((x2 - x1) * (y2 - y1)) / (width * height)
    if ratio >= 0.30:
        return "very close"
    if ratio >= 0.08:
        return "close"
    return "far"


def annotate_detection(det: dict, width: int, height: int) -> dict:
    x1, y1, x2, y2 = det["box"]
    out = dict(det)
    out["zone"] = horizontal_zone(x1, x2, width)
    out["proximity"] = proximity(x1, y1, x2, y2, width, height)
    return out


def build_guidance(detections: list, width: int, height: int, max_items: int = 3) ->
     str:
    """Rank detections and produce a short sentence for text-to-speech."""
    items = [annotate_detection(d, width, height) for d in detections]
    if not items:
        return "No known objects detected. This does not mean the way is clear."
    items.sort(key=lambda d: (-PROX_RANK[d["proximity"]],
                              d["label"] not in HAZARDS,
                              d["zone"] != "center",
                              -d["confidence"]))
    parts = []
    for d in items[:max_items]:
        text = f'{d["label"]} {PHRASE[d["zone"]]}, {d["proximity"]}'
        parts.append(text[:1].upper() + text[1:])
    return ". ".join(parts) + "."


class GuidanceThrottle:
    """Avoid repeating the same message every frame.

    Speak when the message changed (and min_gap seconds have passed) or when
```

```
    repeat_after seconds have passed since the last utterance.
    """

    def __init__(self, min_gap: float = 1.0, repeat_after: float = 8.0, clock=time.
    monotonic):
        self.min_gap, self.repeat_after, self.clock = min_gap, repeat_after, clock
        self.last_message = None
        self.last_time = float("-inf")

    def should_speak(self, message: str) -> bool:
        now = self.clock()
        elapsed = now - self.last_time
        changed = message != self.last_message
        if (changed and elapsed >= self.min_gap) or elapsed >= self.repeat_after:
            self.last_message, self.last_time = message, now
            return True
        return False
```

Listing 7.1: `netra/spatial.py`

The `GuidanceThrottle` class prevents the system from repeating the same sentence every frame. It speaks when the message *changes* (after a minimum gap) or when a repeat interval has passed, and takes an injectable clock so the unit test needs no real waiting.

7.2 Object detection with a small YOLO model

Install the optional dependency (this pulls in PyTorch and is a large download):

```
pip install -r requirements-detect.txt
```

The code loads `models/yolo11n.pt` if it exists; otherwise it asks Ultralytics for `yolo11n.pt`, which is downloaded into the current directory on first use (move it into `models/` afterwards). Verify the current model names and download behaviour in the Ultralytics documentation; they change between releases. "Nano" models trade accuracy for speed, which suits edge devices.

Licensing

Ultralytics software and weights are offered under the AGPL-3.0 licence or a separate commercial licence. The AGPL carries obligations for distribution and for network use. If you plan to distribute a product, read the current terms at the Ultralytics licence page and consider alternatives with permissive licences, or the commercial option.

```
"""Object detection with a small YOLO model, plus spatial guidance.

Ultralytics (and PyTorch) are imported lazily so the rest of the project
works without them. Check the Ultralytics licence before distributing.
"""
import json
import time

import cv2

from .common import BASE, REPORT_OUT, VIDEO_EXT, VIDEO_IN, load_image, rel,
    safe_path, save_image
from .spatial import annotate_detection, build_guidance

_model = None


```

```python
def _get_model():
    global _model
    if _model is None:
        from ultralytics import YOLO
        local = BASE / "models" / "yolo11n.pt"
        _model = YOLO(str(local) if local.is_file() else "yolo11n.pt")
    return _model


def detect_frame(frame, conf: float = 0.35, imgsz: int = 640) -> list:
    """Run the detector on a BGR frame; return label, confidence and pixel box."""
    model = _get_model()
    result = model.predict(frame, conf=conf, imgsz=imgsz, verbose=False)[0]
    dets = []
    for box, c, k in zip(result.boxes.xyxy.tolist(), result.boxes.conf.tolist(),
                         result.boxes.cls.tolist()):
        dets.append({"label": model.names[int(k)], "confidence": round(float(c), 3),
                     "box": [round(v, 1) for v in box]})
    return dets


def draw_detections(frame, dets: list):
    """Draw boxes labelled with class, zone and proximity on a copy of frame."""
    h, w = frame.shape[:2]
    canvas = frame.copy()
    for d in dets:
        a = annotate_detection(d, w, h)
        x1, y1, x2, y2 = (int(v) for v in a["box"])
        cv2.rectangle(canvas, (x1, y1), (x2, y2), (0, 200, 0), 2)
        text = f'{a["label"]} {a["confidence"]:.2f} | {a["zone"]}, {a["proximity"]}'
        cv2.putText(canvas, text, (x1, max(15, y1 - 6)), cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (0, 255, 255), 1, cv2.LINE_AA)
    return canvas


def detect_objects(filename: str, conf: float = 0.35, folder: str = "input") -> dict:
    """Detect objects in an image and describe where they are.

    Args:
        filename: Image filename.
        conf: Confidence threshold from 0.05 to 0.95.
        folder: Either 'input' or 'output'.
    """
    if not 0.05 <= conf <= 0.95:
        raise ValueError("conf must be in [0.05, 0.95].")
    frame = load_image(filename, folder)
    h, w = frame.shape[:2]
    dets = detect_frame(frame, conf)
    out = save_image(draw_detections(frame, dets), f"detected_{filename}")
    return {"objects": [annotate_detection(d, w, h) for d in dets],
            "guidance": build_guidance(dets, w, h),
            "output": rel(out), "parameters": {"conf": conf}}


def guide_video(filename: str, step_seconds: float = 0.5, conf: float = 0.35) ->
    dict:
    """Sample a video, generate guidance per sample and save a JSON timeline.

    Args:
        filename: Video filename inside input/videos.
        step_seconds: Time between analysed frames, 0.1 to 10.
        conf: Confidence threshold from 0.05 to 0.95.
    """
```

```
    if not 0.1 <= step_seconds <= 10:
        raise ValueError("step_seconds must be in [0.1, 10].")
    if not 0.05 <= conf <= 0.95:
        raise ValueError("conf must be in [0.05, 0.95].")
    path = safe_path(VIDEO_IN, filename, VIDEO_EXT)
    cap = cv2.VideoCapture(str(path))
    if not cap.isOpened():
        raise ValueError("Cannot open video.")
    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
    step = max(1, round(fps * step_seconds))
    timeline, latencies, count, last = [], [], 0, None
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        if count % step == 0:
            h, w = frame.shape[:2]
            t0 = time.perf_counter()
            dets = detect_frame(frame, conf)
            latencies.append((time.perf_counter() - t0) * 1000)
            message = build_guidance(dets, w, h)
            if message != last:
                timeline.append({"time_s": round(count / fps, 2), "guidance":
    message})
                last = message
        count += 1
        if len(latencies) >= 2000:
            break
    cap.release()
    report = REPORT_OUT / f"guidance_{path.stem}.json"
    report.write_text(json.dumps(timeline, indent=2))
    mean_ms = sum(latencies) / len(latencies) if latencies else 0.0
    return {"analysed_frames": len(latencies), "mean_latency_ms": round(mean_ms, 1),
            "messages": len(timeline), "report": rel(report), "first_messages":
    timeline[:5]}


def benchmark_detector(filename: str, runs: int = 10) -> dict:
    """Measure mean inference latency and frames per second on one image.

    Args:
        filename: Image filename inside input/images.
        runs: Number of timed runs, 3 to 100.
    """
    if not 3 <= runs <= 100:
        raise ValueError("runs must be in [3, 100].")
    frame = load_image(filename)
    detect_frame(frame)  # warm-up (model load, first-run allocation)
    times = []
    for _ in range(runs):
        t0 = time.perf_counter()
        detect_frame(frame)
        times.append((time.perf_counter() - t0) * 1000)
    mean = sum(times) / len(times)
    return {"runs": runs, "mean_ms": round(mean, 1), "fps": round(1000 / mean, 2),
            "min_ms": round(min(times), 1), "max_ms": round(max(times), 1)}
```

Listing 7.2: `netra/detection.py`

Reading the code.

- `_get_model` loads the model lazily, so importing the agent does not require PyTorch.

- `detect_frame` works on an in-memory frame, which is what a live camera loop needs; `detect_objects` wraps it for files and writes an annotated copy.
- `guide_video` samples a video, builds guidance per sample, records only *changes* in the sentence, and saves a JSON timeline. It also records per-frame latency, which feeds the evaluation in Chapter 10.
- `benchmark_detector` performs a warm-up run before timing, because the first inference includes model loading and allocation.
- A confidence score is a model score, not a probability that the detection is correct.

Tracking (extension). Detection answers "what is in this frame"; tracking asks "which detection corresponds to an object seen earlier". Ultralytics provides `model.track(source=..., tracker="bytetrack.yaml")`. Tracking lets you say "approaching" by watching a box grow across frames, but identities can switch or vanish under occlusion, so treat them as estimates.

7.3 Reading printed text

OCR converts text in an image to characters. The tool upsamples small images, smooths slightly, binarises with Otsu's method, then calls Tesseract, which returns per-word confidences. Language codes follow Tesseract's three-letter convention (`eng`, `tel`, `hin`) and can be combined with +. Install language packs with `apt`, as in Chapter 3.

```
"""Read printed text (signs, labels, medicine boxes) with Tesseract."""
import re

import cv2

from .common import load_image


def read_text(filename: str, language: str = "eng", folder: str = "input") -> dict:
    """Extract printed text from an image with OCR.

    Args:
        filename: Image filename.
        language: Tesseract language code such as 'eng', 'tel', 'hin' or 'eng+tel'.
        folder: Either 'input' or 'output'.
    """
    if not re.fullmatch(r"[a-z]{3}(\+[a-z]{3})*", language):
        raise ValueError("language must look like 'eng' or 'eng+tel'.")
    import pytesseract
    gray = load_image(filename, folder, grayscale=True)
    if max(gray.shape) < 1000:                      # small text benefits from
    upscaling
        gray = cv2.resize(gray, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)
    gray = cv2.GaussianBlur(gray, (3, 3), 0)
    binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)[1]
    try:
        data = pytesseract.image_to_data(binary, lang=language,
                                         output_type=pytesseract.Output.DICT)
    except pytesseract.TesseractNotFoundError as exc:
        raise RuntimeError("Tesseract is not installed: sudo apt install tesseract-
    ocr") from exc
    words, confs = [], []
    for text, conf in zip(data["text"], data["conf"]):
        if text.strip() and float(conf) >= 0:
            words.append(text.strip())
```

```
            confs.append(float(conf))
    mean_conf = sum(confs) / len(confs) if confs else 0.0
    return {"text": " ".join(words), "word_count": len(words),
            "mean_confidence": round(mean_conf, 1),
            "note": "OCR confidence is not accuracy; verify safety-critical text."}
```

Listing 7.3: `netra/ocr_tools.py`

Mean confidence is not accuracy. For safety-critical text such as medicine dosage, the system should tell the user that reading is uncertain and suggest human verification.

7.4 Speech output

```
"""Offline text-to-speech with espeak-ng (writes a WAV file)."""
import hashlib
import re
import shutil
import subprocess

from .common import AUDIO_OUT, rel


def speak_text(text: str, voice: str = "en", words_per_minute: int = 150) -> dict:
    """Convert text to a WAV file using espeak-ng.

    Args:
        text: Sentence to speak, up to 500 characters.
        voice: espeak-ng voice such as 'en', 'hi' or 'te'.
        words_per_minute: Speaking rate from 80 to 300.
    """
    if not text.strip() or len(text) > 500:
        raise ValueError("text must contain 1 to 500 characters.")
    if not re.fullmatch(r"[a-z]{2,3}(-[a-z0-9]+)?", voice):
        raise ValueError("Invalid voice name.")
    if not 80 <= words_per_minute <= 300:
        raise ValueError("words_per_minute must be in [80, 300].")
    exe = shutil.which("espeak-ng")
    if exe is None:
        raise RuntimeError("espeak-ng is not installed: sudo apt install espeak-ng")
    digest = hashlib.sha1(f"{voice}|{words_per_minute}|{text}".encode()).hexdigest()
    [:10]
    out = AUDIO_OUT / f"speech_{digest}.wav"
    subprocess.run([exe, "--stdin", "-v", voice, "-s", str(words_per_minute), "-w",
    str(out)],
                   input=text, text=True, check=True, timeout=30)
    return {"output": rel(out), "characters": len(text)}
```

Listing 7.4: `netra/speech.py`

The text is passed on *standard input* (`--stdin`) rather than as a command-line argument, so text beginning with a dash cannot be misread as an option, and the command is built as a list so no shell is involved. The file name is a hash of the voice, rate and text, so repeated phrases reuse the same WAV. Voices such as `te` (Telugu) and `hi` (Hindi) exist in espeak-ng; judge their intelligibility with real users before relying on them. Neural voices such as Piper may sound more natural; they are also open source but add model downloads.

7.5 Scene description with a vision-language model

A text-only LLM cannot see pixels. A vision-language model (VLM) accepts an image plus a question. Download one with `ollamapullllama3.2-vision` (large; check memory needs) and choose another with `NETRA_VISION`.

```python
"""Scene description with a local vision-language model served by Ollama."""
from .common import VISION_MODEL, load_image, safe_path, IMAGE_EXT, FOLDERS

PROMPT = ("You are helping a blind person understand a photo from a head-mounted camera. "
          "Answer briefly and concretely. Say 'unclear' instead of guessing. "
          "Never state that a path is safe or clear. ")


def describe_scene(filename: str, question: str = "Describe the scene in two sentences.",
                   folder: str = "input") -> dict:
    """Ask a vision-language model about an image.

    Args:
        filename: Image filename.
        question: What to ask about the image (at most 300 characters).
        folder: Either 'input' or 'output'.
    """
    if not question.strip() or len(question) > 300:
        raise ValueError("question must contain 1 to 300 characters.")
    load_image(filename, folder)                        # validates size and decodability
    path = safe_path(FOLDERS[folder], filename, IMAGE_EXT)
    from ollama import chat
    reply = chat(model=VISION_MODEL,
                 messages=[{"role": "user", "content": PROMPT + question,
                            "images": [str(path)]}])
    return {"answer": reply.message.content, "model": VISION_MODEL,
            "note": "Generated description; not a measurement."}
```

Listing 7.5: `netra/vision_tools.py`

The prompt tells the model to say "unclear" rather than guess and never to say a path is safe. A VLM *describes*; it does not *measure*. If the user needs exact positions, use detection; if they need exact text, use OCR.

7.6 A deterministic pipeline

Figure 7.2 shows `navigation_assist`. Because the steps are fixed, it is reproducible and does not depend on the LLM choosing correctly, which is how you would deploy the safety-relevant path on a device.

```python
"""A deterministic pipeline: assess -> enhance (if needed) -> detect -> speak."""
from .detection import detect_objects
from .image_tools import assess_image_quality, enhance_low_light
from .speech import speak_text


def navigation_assist(filename: str, speak: bool = True) -> dict:
    """Run quality check, optional enhancement, object detection and speech.

    Args:
        filename: Image filename inside input/images.
```

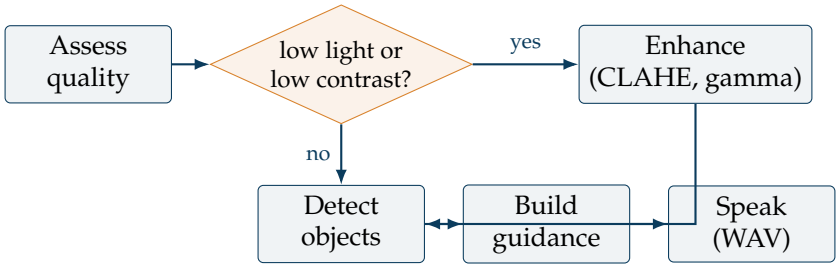


Figure 7.2: The `navigation_assist` pipeline.

```
        speak: If true, also synthesise the guidance sentence to a WAV file.
    """
    steps = []
    quality = assess_image_quality(filename)
    steps.append({"step": "assess_image_quality", "issues": quality["issues"]})
    source, folder = filename, "input"
    if {"low_light", "low_contrast"} & set(quality["issues"]):
        gamma = 1.8 if "low_light" in quality["issues"] else 1.0
        enhanced = enhance_low_light(filename, gamma=gamma)
        steps.append({"step": "enhance_low_light", **enhanced})
        source, folder = enhanced["output"].split("/")[-1], "output"
    result = detect_objects(source, folder=folder)
    steps.append({"step": "detect_objects", "output": result["output"]})
    audio = None
    if speak:
        try:
            audio = speak_text(result["guidance"])["output"]
        except RuntimeError as exc:           # espeak-ng missing: keep the text
result
            steps.append({"step": "speak_text", "skipped": str(exc)})
    return {"guidance": result["guidance"], "objects": result["objects"],
            "annotated": result["output"], "audio": audio, "steps": steps}
```

Listing 7.6: `netra/pipeline.py`

8 Connecting the Tools to the AI Agent

8.1 Function calling

The model receives the user request, a system instruction and the *schemas* of approved functions. For "Enhance `hallway.png` with moderate brightening" it may return a tool call such as `enhance_low_light(filename="hallway.png", gamma=1.8)`. **The Python program, not the model, executes it**, then returns the result so the model can explain what happened.

8.2 The agent controller

```
"""Tool-calling agent: a local LLM chooses approved tools; Python runs them.

Run with:  python -m netra.agent
"""
import json
import time

from .common import BASE, LLM_MODEL, REPORT_OUT
from .detection import benchmark_detector, detect_objects, guide_video
from .image_tools import (assess_image_quality, canny_edges, convert_grayscale,
    denoise_image,
                          enhance_low_light, generate_histogram,
    histogram_equalization,
                          image_info, sharpen_image, threshold_segmentation)
from .metrics import compare_images, simulate_low_light
from .ocr_tools import read_text
from .pipeline import navigation_assist
from .speech import speak_text
from .video_tools import detect_motion, extract_frames, video_info
from .vision_tools import describe_scene

MAX_ROUNDS = 6
LOG_FILE = REPORT_OUT / "agent_log.jsonl"

TOOLS = [
    image_info, assess_image_quality, convert_grayscale, enhance_low_light,
    histogram_equalization, denoise_image, sharpen_image, canny_edges,
    threshold_segmentation, generate_histogram, compare_images, simulate_low_light,
    video_info, extract_frames, detect_motion,
    detect_objects, guide_video, benchmark_detector,
    read_text, speak_text, describe_scene, navigation_assist,
]
TOOL_MAP = {t.__name__: t for t in TOOLS}

SYSTEM_PROMPT = """You are NetraAI, an image and video processing agent for an
    assistive
vision research project (smart glasses for people with visual impairment).
You can only act through the provided tools.

Rules:
- Never claim an operation happened unless a tool result confirms it.
- Never invent filenames, measurements or detections.
- Ask for the filename if the user did not give one.
- Report the tool name, parameters and output location you used.
- Detections are model estimates. Never say that a path or area is safe or clear.
- You are a research aid, not a certified mobility device.
```

```
- Do not run shell commands or Python code."""


def _log(entry: dict) -> None:
    entry["time"] = time.strftime("%Y-%m-%dT%H:%M:%S")
    with LOG_FILE.open("a") as fh:
        fh.write(json.dumps(entry, default=str) + "\n")


def _verify(result) -> None:
    """If a tool claims to have written a file, make sure it exists."""
    if isinstance(result, dict) and isinstance(result.get("output"), str):
        if not (BASE / result["output"]).is_file():
            raise IOError(f"Tool reported {result['output']} but the file does not exist.")


def execute_tool(name: str, arguments: dict) -> dict:
    """Run one allow-listed tool; never raise, so the model can read the error."""
    func = TOOL_MAP.get(name)
    if func is None:
        result = {"error": f"Tool '{name}' is not allowed."}
        status = "rejected"
    else:
        try:
            result = func(**dict(arguments))
            _verify(result)
            status = "ok"
        except Exception as exc:                        # noqa: BLE001 - report to the model
            result, status = {"error": f"{type(exc).__name__}: {exc}"}, "error"
    _log({"tool": name, "arguments": dict(arguments), "status": status, "model": LLM_MODEL})
    return result


def run_query(query: str, history: list | None = None, max_rounds: int = MAX_ROUNDS):
    """Answer one request using at most max_rounds model calls."""
    from ollama import chat
    messages = [{"role": "system", "content": SYSTEM_PROMPT}] + (history or [])
    messages.append({"role": "user", "content": query})
    for _ in range(max_rounds):
        message = chat(model=LLM_MODEL, messages=messages, tools=TOOLS).message
        messages.append(message)
        if not message.tool_calls:
            return message.content, messages[1:]
        for call in message.tool_calls:
            result = execute_tool(call.function.name, call.function.arguments)
            messages.append({"role": "tool", "tool_name": call.function.name,
                             "content": json.dumps(result, default=str)})
    return "Stopped: the tool-call limit was reached.", messages[1:]


def main() -> None:
    print("NetraAI agent. Type 'exit' to quit.")
    history: list = []
    while True:
        query = input("\nYou: ").strip()
        if query.lower() in {"exit", "quit"}:
            break
        if not query:
            continue
        try:
```

```
            answer, history = run_query(query, history[-24:])
            print("\nNetraAI:", answer)
        except Exception as exc:                    # noqa: BLE001
            print("\nError:", exc)


if __name__ == "__main__":
    main()
```

Listing 8.1: `netra/agent.py`

How it works.

1. `TOOLS` is the allow-list. `TOOL_MAP` looks tools up by name. An unknown name is rejected and reported; the model cannot invoke anything else.
2. `execute_tool` *never raises*. Exceptions become `{"error": ...}` so the model can read what went wrong and correct its arguments.
3. `_verify` checks that a file a tool claims to have written actually exists, implementing the rule "verify the output, do not trust the explanation".
4. `run_query` is a *bounded* loop: at most `MAX_ROUNDS` model calls per request, so a confused model cannot loop forever. Several tool calls in one reply are handled in order. This supports multi-step requests such as "enhance, then detect".
5. Every call is appended to `output/reports/agent_log.jsonl` with time, tool, arguments, status and model name.
6. The system prompt forbids claiming unperformed actions, inventing filenames or measurements, and declaring an area safe.

Client-version caveat

The exact message format for tool results can change between Ollama client versions (for example, whether tool messages carry a call identifier). The code was checked only to the extent that all 22 functions convert into valid tool schemas with the `ollama` Python package. Test the full loop against your installed version and the official tool-calling documentation.

8.3 Run the agent

```
cd ~/netra-ai
source .venv/bin/activate
python -m netra.agent
```

`-m` runs a module using package-aware imports, which is why you start from the project root. Example session (wording will vary by model):

```
You: Is hallway.png too dark? Enhance it if so.
NetraAI: I assessed hallway.png (mean intensity 54, flagged low_light) and
ran enhance_low_light with clip_limit 2.0, tile_grid 8, gamma 1.8. The result
is saved at output/images/enhanced_hallway.png.
```

Always open the output file; do not rely only on the model's description of it.

8.4 Hardening checklist

- Strict validation of every argument inside each tool (done) and image-size and video-length limits (done).
- Per-tool timeouts and cancellation for long videos (not yet implemented).
- Unique output names or run identifiers so repeated runs do not overwrite each other.
- Confirmation before overwriting or deleting anything.
- Never use `eval`, `exec` or a shell on model-generated text.

9 Practical Experiments

Copy your own test images into `input/images/` and videos into `input/videos/`. Use your own recordings or images whose licence permits use. Do not use photographs of identifiable people without their consent. Each experiment lists the instruction, the tool the agent should choose, and what to verify. If the model chooses differently, that is a finding worth recording.

#	Instruction	Expected tool	Verify
1	"Give me the properties of `scene.png`."	`image_info`	Width, height, channels, size match `file`.
2	"Is `hallway.png` too dark? Fix it if needed."	`assess_image_quality`, then `enhance_low_light`	Reported mean intensity rises; parameters are reported.
3	"Remove salt-and-pepper noise from `noisy.png`."	`denoise_image` (median)	Isolated specks gone; edges intact. Ask the agent to compare median, Gaussian and bilateral if noise type is unknown.
4	"Read the text on `label.png` in English and Telugu."	`read_text` (`eng+tel`)	Compare against the true text; note the confidence.
5	"Extract one frame every 2 seconds from `walk.mp4`."	`extract_frames`	Count of saved frames $\approx$ duration/2.
6	"Detect objects in `street.png`."	`detect_objects`	Boxes drawn; zone and proximity sensible; sentence spoken.
7	"Analyse `walk.mp4` every 0.5 s."	`guide_video`	JSON timeline; mean latency reported.
8	"Run the full navigation pipeline on `corridor.png`."	`navigation_assist`	Steps list shows enhancement only if needed; WAV exists.

Table 9.1: Agent experiments.

9.1 Controlled low-light experiment (measured)

Real low-light footage has no clean reference, so image-quality metrics such as PSNR and SSIM cannot be computed. The standard remedy is a *controlled* experiment: take a clean image, degrade it in a known way, enhance it, and compare with the original. The script `scripts/run_lowlight_experiment.py` does this with the astronaut photograph bundled with scikit-image, so it runs offline without extra downloads.

```
python scripts/run_lowlight_experiment.py
```

The degradation is $I_{\text{dark}} = 255\,(I/255)^{2.5} + \mathcal{N}(0, 6^2)$ with a fixed random seed. Three enhancement settings are applied to the dark image.

Image	PSNR (dB)	SSIM	Mean intensity
Dark input	13.05	0.625	65.7
CLAHE only ($\gamma = 1.0$)	16.04	0.560	86.7
CLAHE + gamma 1.8	20.88	0.661	121.7
CLAHE + gamma 2.2	19.11	0.668	133.6

Table 9.2: Measured results (one image, one seed). Clip limit 2.0 and an 8×8 tile grid throughout.

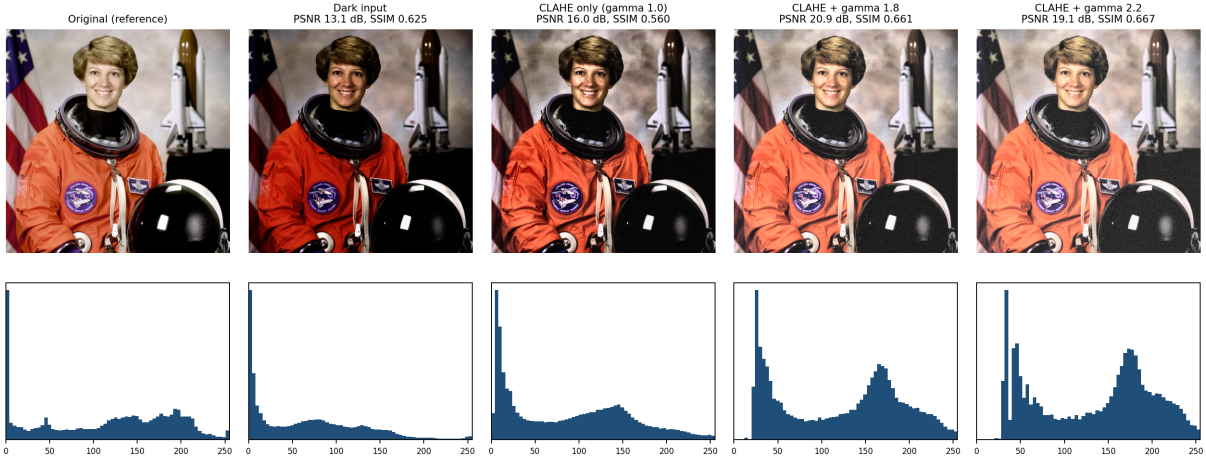


Figure 9.1: Top: original, degraded input, and three enhancements. Bottom: grayscale histograms. PSNR and SSIM are computed against the original on the grayscale versions.

What the numbers do and do not say.

- CLAHE alone *raised* PSNR but *lowered* SSIM below the dark input. A likely explanation is that local contrast stretching also amplifies the added noise, which SSIM's structure term penalises; the evidence here (a single image) is suggestive, not conclusive. This is a good illustration of why one metric is never enough.
- Adding gamma 1.8 gave the highest PSNR; gamma 2.2 gave the highest SSIM but lower PSNR. There is no single "best" setting; it depends on the criterion, and the criterion should match the task (for example detection accuracy downstream).
- The result is for one synthetic degradation of one image. Do not generalise. Repeat over many images, several degradations and multiple seeds, and report mean and spread.
- PSNR and SSIM measure similarity to the original, not usefulness to a blind user or to a detector. The real test of an enhancement step is whether detection recall improves on genuinely dark frames (Chapter 10).

10 Evaluation

10.1 Image-quality metrics

For a reference I and a processed image K of size $M \times N$:

$$\mathrm{MSE} = \frac{1}{MN} \sum_{x,y} \big[I(x,y) - K(x,y) \big]^2, \qquad \mathrm{PSNR} = 10 \log_{10} \frac{\mathrm{MAX}_I^2}{\mathrm{MSE}},$$

with $\mathrm{MAX}_I = 255$ for 8-bit images (PSNR is infinite if the images are identical). SSIM compares luminance, contrast and structure in local windows:

$$\mathrm{SSIM}(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}.$$

10.2 Detection metrics

Match each prediction to a ground-truth box of the same class when their intersection over union exceeds a threshold (commonly 0.5):

$$\mathrm{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|}, \quad P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = \frac{2PR}{P + R}.$$

For an assistive device *recall on hazard classes* and *false alarms per minute* usually matter more than overall accuracy. Report them separately for each lighting condition, because average performance hides failures in the dark.

```
"""Evaluation helpers: image quality (PSNR/SSIM), detection (IoU, precision, recall).
    """
import cv2
import numpy as np
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

from .common import IMAGE_IN, load_image


def compare_images(reference: str, processed: str, processed_folder: str = "output")
     -> dict:
    """Compute PSNR and SSIM between a reference image and a processed image.

    Args:
        reference: Reference filename inside input/images.
        processed: Processed filename.
        processed_folder: Either 'input' or 'output'.
    """
    ref = load_image(reference, "input", grayscale=True)
    proc = load_image(processed, processed_folder, grayscale=True)
    if ref.shape != proc.shape:
        raise ValueError("Image dimensions must match.")
    return {"PSNR_dB": round(float(peak_signal_noise_ratio(ref, proc, data_range
    =255)), 3),
            "SSIM": round(float(structural_similarity(ref, proc, data_range=255)),
    4)}


```

```python
def simulate_low_light(filename: str, gamma: float = 2.5, noise_sigma: float = 6.0,
                       seed: int = 0) -> dict:
    """Create a darkened, noisy copy of an image for controlled experiments.

    The copy is saved to input/images as dark_<filename> so it can be enhanced
    and compared with the original, which acts as the clean reference.

    Args:
        filename: Image filename inside input/images.
        gamma: Darkening strength from 1 to 5.
        noise_sigma: Gaussian noise standard deviation from 0 to 30.
        seed: Random seed for reproducibility.
    """
    if not 1 <= gamma <= 5 or not 0 <= noise_sigma <= 30:
        raise ValueError("gamma must be in [1, 5] and noise_sigma in [0, 30].")
    img = load_image(filename).astype(np.float32) / 255.0
    dark = np.power(img, gamma) * 255.0
    rng = np.random.default_rng(seed)
    dark += rng.normal(0, noise_sigma, dark.shape)
    out_name = f"dark_{filename}"
    if not cv2.imwrite(str(IMAGE_IN / out_name), np.clip(dark, 0, 255).astype(np.
    uint8)):
        raise IOError("Could not save the darkened image.")
    return {"output_input_folder": out_name,
            "parameters": {"gamma": gamma, "noise_sigma": noise_sigma, "seed": seed}}


def iou(a, b) -> float:
    """Intersection over union of two [x1, y1, x2, y2] boxes."""
    ix = max(0.0, min(a[2], b[2]) - max(a[0], b[0]))
    iy = max(0.0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / union if union > 0 else 0.0


def precision_recall(predictions: list, truths: list, iou_threshold: float = 0.5) ->
     dict:
    """Greedy one-to-one matching of predictions to ground truth.

    Each item is {"label": str, "box": [x1, y1, x2, y2]}; predictions may carry
    a "confidence" used to order the matching.
    """
    preds = sorted(predictions, key=lambda p: -p.get("confidence", 1.0))
    used, tp = set(), 0
    for p in preds:
        best, best_iou = None, iou_threshold
        for i, g in enumerate(truths):
            if i in used or g["label"] != p["label"]:
                continue
            value = iou(p["box"], g["box"])
            if value >= best_iou:
                best, best_iou = i, value
        if best is not None:
            used.add(best)
            tp += 1
    fp, fn = len(preds) - tp, len(truths) - tp
    prec = tp / (tp + fp) if tp + fp else 0.0
    rec = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * prec * rec / (prec + rec) if prec + rec else 0.0
    return {"tp": tp, "fp": fp, "fn": fn, "precision": round(prec, 3),
            "recall": round(rec, 3), "f1": round(f1, 3)}
```

Listing 10.1: `netra/metrics.py`

`compare_images` takes the reference from `input/` and the processed image from `output/` (or `input/`); `simulate_low_light` creates the controlled degradation; `iou` and `precision_recall` are pure functions with unit tests. Greedy matching in descending confidence order is a simplification of the full mAP protocol; use the evaluation tools in your detection framework for published numbers.

10.3 Task-level and system-level measures

Measure	How to obtain it	Why it matters
Hazard recall, precision	Annotate 200+ frames per condition; run `precision_recall`	Missed hazards and false alarms.
Zone accuracy	Compare predicted left/center/right with annotated side	Wrong side is worse than no message.
Proximity calibration	Place objects at measured distances; tabulate predicted class vs. true distance per class	Validates or replaces the area heuristic.
End-to-end latency	Time capture → detection → sentence → audio start	Stale guidance is misleading.
Throughput (FPS)	`benchmark_detector` on the target device	Feasibility on edge hardware.
False alarms per minute	Run on hazard-free footage	Alert fatigue.
Speech intelligibility	Listening tests with target users and languages	Usability.
Enhancement value	Detection recall with vs. without enhancement on real dark frames	The real test of Chapter 5 enhancement.

Table 10.1: Evaluation plan for an assistive-vision prototype.

Stage	Latency budget (design target to verify, not a measurement)
Capture and decode	as short as the camera allows
Quality check and enhancement	small compared with inference
Detection	dominant cost; depends on model, input size, device
Guidance sentence	negligible
Speech synthesis start	short for espeak-ng; longer for neural voices

Measure each stage separately on the target device, set an end-to-end target with your mobility instructors (how stale can a message be before it misleads?), and report percentiles, not only the mean.

10.4 Testing and reproducibility

The test suite checks path rejection, invalid arguments, quality flags, an enhancement improving PSNR on a darkened synthetic image, video sampling and motion detection on a generated clip, spatial rules, the throttle, IoU and precision/recall, and detection post-processing with a stand-in model. Excerpt:

```python
from netra.spatial import GuidanceThrottle, build_guidance, horizontal_zone,
    proximity


def test_zones():
    assert horizontal_zone(0, 100, 900) == "left"
    assert horizontal_zone(400, 500, 900) == "center"
    assert horizontal_zone(800, 890, 900) == "right"


def test_proximity_levels():
    assert proximity(0, 0, 600, 600, 1000, 1000) == "very close"   # 36% of the
    frame
    assert proximity(0, 0, 300, 300, 1000, 1000) == "close"        # 9%
    assert proximity(0, 0, 100, 100, 1000, 1000) == "far"


def test_guidance_prefers_closer_object():
    dets = [
        {"label": "chair", "confidence": 0.9, "box": [0, 0, 100, 100]},
        {"label": "person", "confidence": 0.8, "box": [300, 100, 700, 900]},
    ]
    text = build_guidance(dets, 1000, 1000)
    assert text.startswith("Person ahead, very close")


def test_empty_guidance_never_claims_clear_path():
    text = build_guidance([], 640, 480)
    assert "does not mean the way is clear" in text


def test_throttle():
    t = [0.0]
    th = GuidanceThrottle(min_gap=1.0, repeat_after=8.0, clock=lambda: t[0])
    assert th.should_speak("A")
    t[0] = 0.5
    assert not th.should_speak("B")     # changed but too soon
    t[0] = 1.5
    assert th.should_speak("B")
    t[0] = 3.0
    assert not th.should_speak("B")     # unchanged
    t[0] = 10.0
```

Listing 10.2: `tests/test_spatial.py`

For research reproducibility, record with each result: input file checksum (`sha256sum`), model name and version, library versions (`pipfreeze`), all parameters (the tools return them), a timestamp, and the output checksum. The agent log already stores tool, arguments, status and model.

11 Interfaces: Streamlit and the Live Loop

11.1 Streamlit

```
streamlit run app.py
```

Streamlit turns the Python script into a local web app, usually at `http://localhost:8501`. Under WSL, open that address in the Windows browser.

```
"""Streamlit interface:  streamlit run app.py"""
import cv2
import streamlit as st

from netra.agent import run_query
from netra.common import BASE, IMAGE_IN
from netra.image_tools import assess_image_quality, enhance_low_light

st.set_page_config(page_title="NetraAI", layout="wide")
st.title("NetraAI: assistive-vision image and video processing")
st.caption("Research prototype. Not a certified mobility or safety device.")

upload = st.sidebar.file_uploader("Upload an image", type=["png", "jpg", "jpeg", "
    bmp"])
filename = None
if upload:
    filename = f"upload_{abs(hash(upload.name)) % 10**8}.{upload.name.rsplit('.', 1)
    [-1].lower()}"
    (IMAGE_IN / filename).write_bytes(upload.getvalue())
    st.sidebar.image(cv2.cvtColor(cv2.imread(str(IMAGE_IN / filename)), cv2.
    COLOR_BGR2RGB))
    st.sidebar.write(f"Stored as `{filename}`")

tab_tools, tab_chat = st.tabs(["Tools", "Chat agent"])

with tab_tools:
    if not filename:
        st.info("Upload an image in the sidebar.")
    else:
        c1, c2 = st.columns(2)
        if c1.button("Assess quality"):
            st.json(assess_image_quality(filename))
        if c2.button("Enhance low light"):
            res = enhance_low_light(filename, gamma=1.8)
            st.json(res)
            st.image(cv2.cvtColor(cv2.imread(str(BASE / res["output"])), cv2.
    COLOR_BGR2RGB))
        if st.button("Detect objects and speak guidance"):
            from netra.pipeline import navigation_assist
            res = navigation_assist(filename)
            st.subheader(res["guidance"])
            st.image(cv2.cvtColor(cv2.imread(str(BASE / res["annotated"])), cv2.
    COLOR_BGR2RGB))
            if res["audio"]:
                st.audio(str(BASE / res["audio"]))

with tab_chat:
    if "history" not in st.session_state:
        st.session_state.history = []
    for m in st.session_state.history:
```

```
        role = m["role"] if isinstance(m, dict) else m.role
        text = m["content"] if isinstance(m, dict) else m.content
        if role in {"user", "assistant"} and text:
            st.chat_message(role).write(text)
    prompt = st.chat_input("e.g. Assess dark_scene.png and enhance it if needed")
    if prompt:
        st.chat_message("user").write(prompt)
        with st.spinner("Working..."):
            answer, st.session_state.history = run_query(prompt, st.session_state.history[-24:])
        st.chat_message("assistant").write(answer)
```

Listing 11.1: `app.py`

Two tabs are provided: direct tool buttons (assess, enhance, detect and speak) and a chat tab that calls the agent and keeps a short history. For a deployed app, use unique server-generated filenames (the example derives one), enforce upload size limits, authenticate users, and do not trust uploaded filenames.

11.2 Live guidance from a camera or a video file

```
python -m netra.live --source street.mp4 --max-seconds 30
python -m netra.live --source 0              # first webcam (native Ubuntu / Pi)
```

```
"""Near-real-time guidance from a webcam index or a video file.

    python -m netra.live --source 0                 # webcam
    python -m netra.live --source street.mp4        # file in input/videos
"""
import argparse
import shutil
import subprocess
import time

import cv2

from .common import VIDEO_EXT, VIDEO_IN, safe_path
from .detection import detect_frame
from .spatial import GuidanceThrottle, build_guidance


def run(source: str, step: float, conf: float, speak: bool, max_seconds: float) -> None:
    target = int(source) if source.isdigit() else str(safe_path(VIDEO_IN, source, VIDEO_EXT))
    cap = cv2.VideoCapture(target)
    if not cap.isOpened():
        raise SystemExit("Cannot open the video source.")
    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0
    skip = max(1, round(fps * step))
    throttle, exe, proc = GuidanceThrottle(), shutil.which("espeak-ng"), None
    start, count = time.monotonic(), 0
    try:
        while time.monotonic() - start < max_seconds:
            ok, frame = cap.read()
            if not ok:
                break
            count += 1
            if count % skip:
                continue
```

```
            h, w = frame.shape[:2]
            t0 = time.perf_counter()
            message = build_guidance(detect_frame(frame, conf), w, h)
            ms = (time.perf_counter() - t0) * 1000
            if throttle.should_speak(message):
                print(f"[{time.monotonic() - start:6.1f}s | {ms:5.0f} ms] {message}"
    )
                if speak and exe and (proc is None or proc.poll() is not None):
                    proc = subprocess.Popen([exe, "--stdin"], stdin=subprocess.PIPE,
     text=True)
                    proc.stdin.write(message)
                    proc.stdin.close()
    except KeyboardInterrupt:
        pass
    finally:
        cap.release()


if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--source", default="0")
    ap.add_argument("--step", type=float, default=0.5, help="seconds between
    analysed frames")
    ap.add_argument("--conf", type=float, default=0.35)
    ap.add_argument("--no-speech", action="store_true")
    ap.add_argument("--max-seconds", type=float, default=60.0)
    a = ap.parse_args()
    run(a.source, a.step, a.conf, not a.no_speech, a.max_seconds)
```

Listing 11.2: `netra/live.py`

The loop analyses a frame every `--step` seconds, builds the guidance sentence, and lets `GuidanceThrottle` decide whether to speak. Speech runs in a separate process so it never blocks capture, and a new utterance is not started while the previous one is still playing. Recorded video files make the loop testable in WSL and repeatable for evaluation.

12 Edge Deployment

A wearable needs the detection and guidance path to run on a small board. The LLM agent is a development and research tool; on the device you would normally run only the deterministic pipeline.

12.1 Design levers

- **Input size.** Inference cost grows roughly with pixel count. Try `imgsz=320` before 640 and measure recall loss.
- **Frame skipping.** Analyse two frames per second, not thirty; walking speed does not need more for coarse guidance.
- **Model format.** Export the model to a lighter runtime (ONNX, NCNN, TFLite) and benchmark each on the device.
- **Throttle speech.** Avoid repetition (`GuidanceThrottle`); fewer, better messages beat constant chatter.
- **Thermals and power.** Sustained inference throttles small boards; measure over minutes, not seconds, and measure battery life.
- **Audio path.** Bone-conduction or open-ear Bluetooth headphones keep ambient sound audible; account for Bluetooth latency.

12.2 Exporting a model

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
model.export(format="onnx", imgsz=320)      # also e.g. format="ncnn" or "tflite"
```

Supported formats and their runtime requirements change; consult the Ultralytics export documentation for your version. Verify that detection quality after export matches the original on your own validation set.

12.3 On the Raspberry Pi

1. Install Raspberry Pi OS or Ubuntu, then repeat the apt and virtual-environment steps of Chapter 3 (skip Ollama; it is too heavy for the smallest boards).
2. Run `python -m pytest -q` to check the core pipeline on ARM.
3. Run `python -m netra.live --source 0 --step 0.5` and note the printed latency per analysed frame.
4. Record latency distribution, memory use, temperature and battery drain over a 30-minute run.

Small boards such as the Zero 2 W have very limited memory and CPU. If your measured latency does not meet your target, options are a faster board, a smaller input size, a lighter export format, or an accelerator. Decide from measurements, not assumptions.

13 Reliability, Safety, Privacy and Ethics

13.1 Failure modes and safeguards

Problem	Safeguard
Missed hazard	Report recall per class and condition; never claim the way is clear; keep traditional aids.
False alarms	Confidence threshold, throttle, per-class tuning; measure alarms per minute.
Wrong side or distance	Calibrate zones and proximity; restrict claims to calibrated classes.
Motion blur, glare, darkness	Quality assessment, enhancement, and a spoken "image unclear" message when quality is too low.
Wrong tool or parameters	Allow-list, in-tool validation, bounded rounds, logged calls.
Model invents results	Rule "never claim without a tool result"; verify output files exist.
Path traversal or arbitrary execution	Plain filenames only; no shell, `eval` or `exec` on model text.
Resource exhaustion	Image-size and video-length limits; add timeouts and concurrency limits.
Overwriting data	Prefix outputs; keep originals read-only in practice.
Misreading text	Report OCR confidence; recommend human verification for medicine or safety text.

13.2 Operational rules

1. Keep the Ollama API bound to a trusted local interface unless a secure network design is deliberate.
2. Restrict file access to the project directories.
3. Record the selected algorithm and all parameters; preserve originals.
4. Ask for confirmation before destructive or expensive operations.
5. Set a maximum number of tool-call rounds (done) and log errors.

13.3 Privacy

A wearable camera captures bystanders. Process locally, do not store frames longer than needed, avoid uploading, and tell people nearby when recording. Do not commit personal images or video to GitHub (the `.gitignore` excludes `input/` and `output/`). Check local law on recording in public and private spaces.

13.4 Working with users

Involve visually impaired users and orientation-and-mobility professionals from the start, not only at the end. Ask what information they want, how often, and in which language and voice. An evaluation with participants needs institutional ethical approval and informed consent. Avoid describing the system as improving "safety" until you have evidence.

14 Publishing the Project on GitHub

14.1 Before the first commit

- Replace the name placeholder in `LICENSE` and the repository URL in `README.md` and in `docs/netra_tutorial.tex`.
- Confirm `.gitignore` excludes `.venv/`, `input/`, `output/`, model weights and LaTeX build files.
- Make sure no secrets, personal photographs or recordings are in the tree.
- Decide the licence. MIT is permissive for your own code, but dependencies keep their own licences; in particular read the Ultralytics terms before distributing anything that depends on it.

14.2 Create the local repository

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
cd ~/netra-ai
git init -b main
git add .
git status
git commit -m "Initial commit: NetraAI assistive-vision agent"
```

`gitinit-bmain` creates a repository whose first branch is `main`. `gitadd.` stages everything not ignored. **Read the output of `gitstatus`**: if `input/`, `output/` or any `.pt` file appears, fix `.gitignore` before committing. `gitcommit-m` records the snapshot with a message.

14.3 Create the GitHub repository

On github.com choose *New repository*, name it (for example `netra-ai`), and create it *empty*: no README, licence or `.gitignore`, because you already have them locally.

14.4 Authenticate

An SSH key is convenient for WSL.

```
ssh-keygen -t ed25519 -C "you@example.com"
cat ~/.ssh/id_ed25519.pub
```

Copy the printed public key into GitHub under *Settings* → *SSH and GPG keys* → *New SSH key*, then test:

```
ssh -T git@github.com
```

Never share the private key (`id_ed25519` without `.pub`). Alternatively use HTTPS with a personal access token or the GitHub CLI.

14.5 Push

```
git remote add origin git@github.com:<your-user>/netra-ai.git
git push -u origin main
```

`remoteadd` names the remote `origin`; `push-u` uploads `main` and remembers the upstream so later you can simply run `gitpush`. After the push, the workflow in `.github/workflows/ci.yml` runs the test suite on every push and pull request; check the *Actions* tab for a green result.

14.6 Good practice afterwards

- Work on branches (`gitswitch-cfeature/ocr-confidence`) and merge through pull requests.
- Tag releases: `gittag-av0.1.0-m"Firstrelease"` then `gitpushoriginv0.1.0`.
- Put the compiled tutorial PDF in `docs/` so readers can open it from the repository, and keep experiment outputs small. For larger sample media use Git LFS (`gitlfsinstall`; `gitlfstrack"*.mp4"`) or link to a dataset host, and check the licence of any dataset you redistribute.
- Add a short "Validation status" section to the README listing what you tested on which hardware, as this project does.
- If you ever commit a secret by mistake, rotating the secret is required; deleting the file in a later commit does not remove it from history.

14.7 Building this document

```
sudo apt install -y texlive-latex-extra texlive-fonts-recommended latexmk
cd docs
latexmk -pdf netra_tutorial.tex
```

The listings are included from the source files, so rebuilding the PDF after changing the code updates the document. Run `pythonscripts/run_lowlight_experiment.py` first if you want to regenerate the figure.

15 Roadmap and Hardware

Phase	Activity	Deliverable
1	WSL2/Ubuntu environment, tests passing	Reproducible setup
2	Image-quality tools and controlled experiments	Enhancement study with metrics
3	Video tools, recorded walking footage (with consent)	Annotated evaluation set
4	Detection, zones, proximity calibration	Per-class recall and calibration table
5	OCR and speech in English, Telugu and Hindi	Intelligibility results with users
6	Agent with Ollama, logging, bounded loop	Verified tool calling on your client version
7	Edge export and Raspberry Pi benchmark	Latency, thermal and battery data
8	User study with ethical approval	Findings and design changes
9	Tracking, depth sensing, haptic output	V2 prototype

Component	Suggested configuration
Development PC	Modern Intel Core i5/i7 or AMD Ryzen 5/7; 16 GB RAM minimum, 32 GB more comfortable for local models; about 100 GB free disk.
GPU (optional)	NVIDIA GPU with current driver; CPU is enough for this tutorial.
Camera	USB webcam or Raspberry Pi camera; fixed mounting for repeatable tests.
Edge board	Raspberry Pi-class board; choose by measured latency, not by price alone.
Audio	Open-ear or bone-conduction headphones.
Network	Internet for initial downloads; local processing afterwards.

Table 15.1: Planning guidelines, not guaranteed minimums. Actual needs depend on model size, quantisation, resolution and concurrent jobs.

16 Official Documentation and Further Reading

Software	Official resource
Ollama	https://docs.ollama.com/
Ollama tool calling	https://docs.ollama.com/capabilities/tool-calling
Ollama Python client	https://github.com/ollama/ollama-python
OpenCV	https://docs.opencv.org/
scikit-image	https://scikit-image.org/docs/stable/
Ultralytics	https://docs.ultralytics.com/ and https://www.ultralytics.com/license
Tesseract	https://tesseract-ocr.github.io/
espeak-ng	https://github.com/espeak-ng/espeak-ng
FFmpeg	https://ffmpeg.org/documentation.html
Streamlit	https://docs.streamlit.io/
WSL	https://learn.microsoft.com/windows/wsl/
GitHub docs	https://docs.github.com/

Conclusion

NetraAI separates concerns: a language model interprets requests and chooses from approved tools; deterministic libraries do the processing; a small, testable guidance layer turns detections into short spoken sentences without claiming more than the evidence supports. Start with the parts you can verify today, namely quality assessment, enhancement, metrics and the test suite. Then add detection, speech and the agent loop one at a time, and measure each on your own data and hardware. Treat the system as a research tool, involve the people it is meant to serve, and report limitations as carefully as results.